

CE E UN INDEX

O structură de date **separată** de tabel care ține evidența unde se află fiecare valoare — ca un cuprins dintr-o carte.

Tabel (heap) — datele efective, nesortate:

Pagina 1: [Ion, Maria, Alex, Ion, Andrei]

Pagina 2: [Maria, Ion, Elena, Alex, Ion]

Index pe name — structură SEPARATĂ, sortată:

"Alex"	→ Pagina 1 rând 3	
"Alex"	→ Pagina 2 rând 4	
"Andrei"	→ Pagina 1 rând 5	
"Elena"	→ Pagina 2 rând 3	
"Ion"	→ Pagina 1 rând 1	
"Ion"	→ Pagina 1 rând 4	
...		

Indexul e sortat — tabelul NU.

Indexul NU mută datele din tabel.

CÂND E UTIL UN INDEX

✅ Util:

Tabele mari (mii/milioane de rânduri)

Coloane căutate frecvent în WHERE

Coloane folosite în JOIN (FK-uri)

Coloane cu selectivitate mare (multe valori distincte)

❌ Nu e util: Tabele mici (Seq Scan e oricum rapid) Coloane cu selectivitate mică (status cu 5 valori)

Tabele cu multe scrieri (INSERT/UPDATE/DELETE actualizează toți indecșii)

CUM SE CREEAZĂ

-- Index simplu

CREATE INDEX idx_numa ON tabel(coloana);

-- Index compus

CREATE INDEX idx_numa ON tabel(col1, col2, col3);

-- Covering index (elimină heap fetch)

CREATE INDEX idx_numa ON tabel(coloana) INCLUDE (col1, col2);

-- Partial index (doar un subset de date)

CREATE INDEX idx_numa ON tabel(coloana) WHERE conditie;

-- Index pe expresie (pentru funcții)

CREATE INDEX idx_numa ON tabel(LOWER(coloana));

ORDINEA COLOANELOR ÎN INDEX COMPUS

Pasul 1 — CE coloane pun în index?

→ lista profesorului:

1. FROM + JOIN → coloanele de JOIN (FK-uri)

2. WHERE → coloanele filtrate

3. GROUP BY → coloanele de grupare

4. SELECT → INCLUDE (covering)

5. ORDER BY → sortare ASC/DESC

Pasul 2 — În CE ORDINE le pun?

→ tipul de operator:

1. Coloanele cu = → primele

2. Coloanele cu >, <, BETWEEN → la mijloc

3. Coloanele cu <>, != → ultimele

Pasul 3 — În cadrul aceluiași operator?

→ selectivitatea:

→ cea mai selectivă primaRegula prefixului:

Index (a, b, c) funcționează pentru:

WHERE a = ... 

WHERE a = ... AND b = ... 

WHERE a = ... AND b = ... AND c = ... 

NU funcționează pentru:

WHERE b = ... 

WHERE c = ... 

WHERE b = ... AND c = ... 

INDEX KILLERS — CÂND INDEXUL NU E FOLOSIT

1. Funcții pe coloană:

WHERE EXTRACT(YEAR FROM date) = 2026

→ Soluție: index pe expresie sau rescrie query-ul

2. LOWER/UPPER:

WHERE LOWER(name) = 'popescu'

→ Soluție: CREATE INDEX ON tabel(LOWER(name))

3. OR pe coloane diferite:

WHERE total > 500 OR status = 'pending'

→ Soluție: UNION ALL

4. LIKE cu % la început:

WHERE email LIKE '%yahoo.com'

→ Soluție: B-Tree nu poate, doar GIN index

CE E HEAP-UL

Tabelul propriu-zis — locul unde sunt stocate efectiv toate rândurile pe disc, organizate în pagini de 8KB.

Heap (tabel orders):

Pagina 1: rândurile 1-50	
Pagina 2: rândurile 51-100	
Pagina 3: rândurile 101-150	
...	
Pagina 200000: rândurile 9.9M+	

Rândurile NU sunt sortate — sunt în ordinea inserării.

CE E HEAP FETCH

Pasul în care PostgreSQL merge din index în tabel (heap) ca să citească coloanele care NU sunt în index:

Fără INCLUDE:

1. Index scan → găsesc customer_id=42 pe Pagina 78 (rapid)

2. Heap fetch → merg pe Pagina 78, citesc status, total (lent)

= 2 operații

Cu INCLUDE (status, total):

1. Index scan → găsesc customer_id=42 ȘI am deja status, total

= 1 operație, heap fetch eliminat ✓

SCANĂRI

Seq Scan = citește TOATĂ tabela, pagină cu pagină

→ lent pe tabele mari

→ apare când lipsește index

Index Scan = caută în index, apoi heap fetch

→ rapid, dar face 2 pași

→ apare cu index normal

Index Only Scan = totul găsit în index, fără heap fetch

→ cel mai rapid

→ apare cu Covering Index (INCLUDE)

JOIN-URI ÎN PLANUL DE EXECUȚIE

Nested Loop = pentru fiecare rând din A, caută în B

→ bun când A e mic și B are index

→ lent când ambele sunt mari

Hash Join = construiește hash pe tabelul mic,

apoi scanează tabelul mare

→ bun pentru tabele mari fără index

Merge Join = sortează ambele tabele, apoi interclasează

→ bun când datele sunt deja sortate

Operații pe rezultate

Sort = sortare explicită (**ORDER BY** fără index)

HashAggregate = **GROUP BY** (calculează **COUNT**, **AVG**, **SUM** per grup)

Filter = aplică **WHERE**

Append = **UNION / UNION ALL** (lipește rezultatele)

CE VREI SĂ VEZI VS CE NU VREI

 **BUN:**

Index Scan / Index Only Scan

Merge Join

 **DEPINDE:**

Hash Join (ok pe tabele mari fără index)

Nested Loop (ok dacă tabelul intern e mic)

 **RĂU (pe tabele mari):**

Seq Scan → probabil lipsește un index

*Sort → lipsește index pe **ORDER BY***

CONSECINȚELE INDECSILOR

 **Avantaje:**

*Citiri mult mai rapide (**SELECT**)*

Elimină Seq Scan pe tabele mari

✗ Dezavantaje:

INSERT mai lent (actualizează toți indecșii)

UPDATE mai lent (actualizează indecșii afectați)

DELETE mai lent

Ocupă spațiu suplimentar pe disc